

CE802 2019 - Worked Solutions

Machine Learning and Data Mining

Approach used in this document: each question is solved from first principles. Computational parts show the formula used, the substitution of the numbers, and the final interpretation. Conceptual parts explain the reasoning rather than only giving a one-line answer.

Source paper: University of Essex, CE802-7-AU (2019).

Note on Question 1(b): support vectors are identified visually from the plotted decision-function contours. In the discussion below, the standard SVM convention is used: support vectors are the training points lying on the margin or inside the margin.

Question 1 - Support Vector Machines

1(a) Why a support vector classifier can be regarded as an instance-based learner

A support vector classifier predicts the label of a new point x by comparing x with selected training examples. In the dual representation, the classifier has the form

$$f(x) = \text{sign}(\sum_i \alpha_i y_i K(x_i, x) + b).$$

Here x_i are training instances, y_i their class labels, $\alpha_i \geq 0$ are learned weights, $K(x_i, x)$ is a similarity measure (the kernel), and only those training points with $\alpha_i > 0$ remain active. Therefore the model behaves like an instance-based learner: it does not use every training example equally, but it does keep a subset of examples and compares a test point directly against them.

- Similarity measure used: the kernel $K(x_i, x)$. With a linear kernel this is the inner product $x_i^T x$; with a polynomial kernel it is a powered inner product; with an RBF kernel it is Gaussian similarity $\exp(-\gamma \|x_i - x\|^2)$.
- Instances retained: only the support vectors, i.e. the training examples that lie on the margin or inside it. All other points have $\alpha_i = 0$ and drop out of the prediction rule.
- Contribution to prediction: each support vector votes according to $\alpha_i y_i K(x_i, x)$. Nearby or similar points contribute more under the kernel; the sign of the total weighted sum determines the class.

1(b)(i) Identifying the support vectors from Figure 1.1

In an SVM plot, the central contour $f(x)=0$ is the decision boundary. The contours $f(x)=+1$ and $f(x)=-1$ mark the two margins. Support vectors are the points that either touch these margins or lie inside them. Reading the figure in that way, the active points are c, e, g, k and m.

- c is a +1 example that sits on the wrong side of the negative margin, so it must be active.
- e is the clearest -1 point on the -1 contour.
- g is a -1 point that lies between the decision boundary and the negative margin, so it is inside the margin.
- k is the clearest +1 point on the +1 contour.
- m is a +1 point very close to the right-hand margin and is also active.

Points such as a, b, d, f, h, i, j, l and n are farther away from the active margins and therefore do not materially determine the boundary.

1(b)(ii) Effect of adding a new +1 example at [1.0, 1.0]

The new point [1.0, 1.0] would lie well inside the existing +1 region, near other positive examples already far from the boundary. Because it is not close to the separating contour, it would probably not become a support vector. Consequently the decision boundary would remain essentially unchanged, and the margin would also stay almost the same. At most, one might expect a very small outward relaxation of the +1 side in the upper-right corner, but no qualitative change in the classifier.

1(c) RBF kernel as a function of squared distance and the analogy with k-NN

For an RBF kernel, $K_{\text{RBF}}(x, z) = \exp(-\gamma \|x - z\|^2)$. If we set $d = \|x - z\|^2$, then $K(d) = \exp(-\gamma d)$. This always starts at 1 when $d=0$ and then decays monotonically towards 0 as d grows.

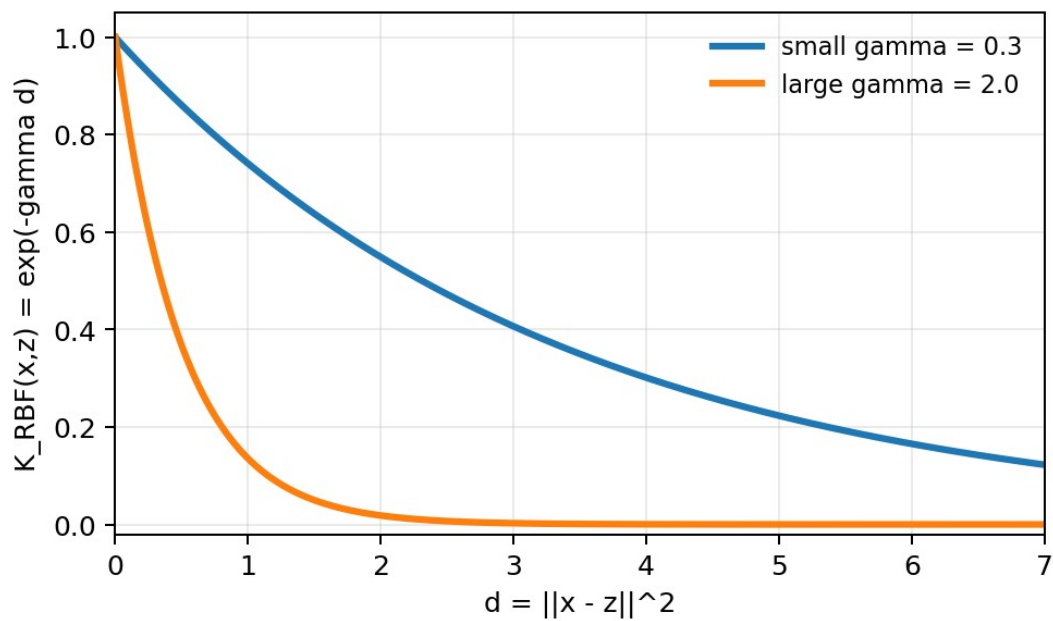


Illustration: the larger gamma is, the faster similarity decays with distance.

- Small gamma: slow decay. Even fairly distant points still have noticeable similarity, so the boundary is smoother and more global. This is analogous to using a large k in k -NN.
- Large gamma: rapid decay. Only very nearby points matter, so the classifier becomes highly local and can bend around the data more aggressively. This is analogous to using a small k in k -NN.

Bias-variance interpretation.

- Large gamma in an SVM behaves like small k in k -NN: low bias, high variance, higher risk of overfitting.
- Small gamma in an SVM behaves like large k in k -NN: high bias, low variance, higher risk of underfitting.

1(d) 0-1 loss versus hinge loss

For a binary classifier with margin $z = y f(x)$, the two loss functions are

0-1 loss: $L_{01}(z) = 1$ if $z \leq 0$, otherwise 0.

hinge loss: $L_{\text{hinge}}(z) = \max(0, 1 - z)$.

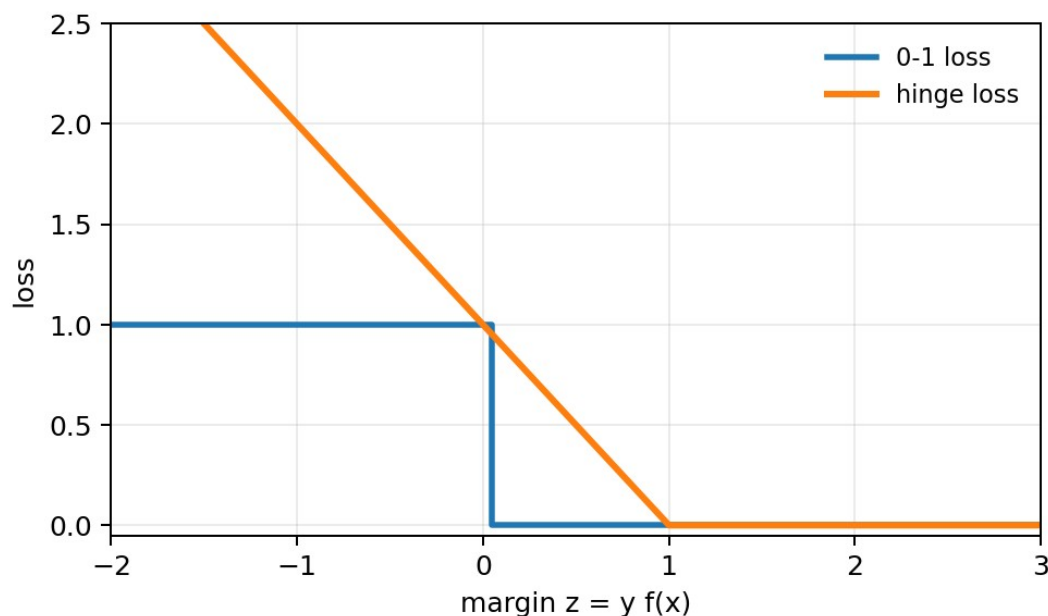


Illustration: the 0-1 loss is a step; the hinge loss is a convex upper bound.

Three disadvantages of the 0-1 loss compared with the hinge loss are:

- It is discontinuous and non-convex, so optimization is difficult and standard gradient-based convex optimization machinery does not apply cleanly.
- It gives the same loss 0 to every correctly classified point, regardless of whether the point is barely correct or has a large safety margin. Hence it does not encourage large-margin solutions.
- Its piecewise-constant shape creates plateaus and combinatorial search difficulties. The hinge loss is easier to optimize and still penalizes borderline points, which is exactly what a maximum-margin classifier needs.

Question 2 - Clustering

2(a) Supervised versus unsupervised learning

Supervised learning uses labelled examples (x, y) . The goal is to learn a mapping from input features to known target values such as classes or real-valued outputs. Unsupervised learning uses unlabelled data only; the goal is to discover structure such as groups, latent components, or low-dimensional representations.

- Examples of supervised algorithms: support vector machines, decision trees, logistic regression, k-NN classifiers.
- Examples of unsupervised algorithms: k-means clustering, hierarchical clustering, Gaussian mixture modelling, principal component analysis.

2(b) Running k-means for the six points $p=(-3,2)$, $q=(1,2)$, $r=(11,8)$, $s=(7,8)$, $t=(-1,11)$, $u=(9,-1)$

Initial seeds: $\mu_1(0)=t=(-1,11)$ and $\mu_2(0)=u=(9,-1)$. To compare distances, it is enough to compare squared Euclidean distances because $\text{sqrt}(\cdot)$ preserves ordering.

Iteration 1

Point	d^2 to $\mu_1 = (-1,11)$	d^2 to $\mu_2 = (9,-1)$	Assigned cluster
p	85	153	C1
q	85	73	C2

r	153	85	C2
s	73	85	C1
t	0	244	C1
u	244	0	C2

New centres from the mean update: $\mu_1 = ((-3+7-1)/3, (2+8+11)/3) = (1, 7)$, $\mu_2 = ((1+11+9)/3, (2+8-1)/3) = (7, 3)$.

Iteration 2

Point	d^2 to $\mu_1 = (1,7)$	d^2 to $\mu_2 = (7,3)$	Assigned cluster
p	41	101	C1
q	25	37	C1
r	101	41	C2
s	37	25	C2
t	20	128	C1
u	128	20	C2

New centres from the mean update: $\mu_1 = ((-3+1-1)/3, (2+2+11)/3) = (-1, 5)$, $\mu_2 = ((11+7+9)/3, (8+8-1)/3) = (9, 5)$.

Iteration 3

Point	d^2 to $\mu_1 = (-1,5)$	d^2 to $\mu_2 = (9,5)$	Assigned cluster
p	13	153	C1
q	13	73	C1
r	153	13	C2
s	73	13	C2
t	36	136	C1
u	136	36	C2

New centres from the mean update: $\mu_1 = ((-3+1-1)/3, (2+2+11)/3) = (-1, 5)$, $\mu_2 = ((11+7+9)/3, (8+8-1)/3) = (9, 5)$.

Since the assignments in Iteration 3 are identical to those in Iteration 2, the algorithm has converged. The final partition is $C1=\{p,q,t\}$ and $C2=\{r,s,u\}$, with final centroids $(-1,5)$ and $(9,5)$.

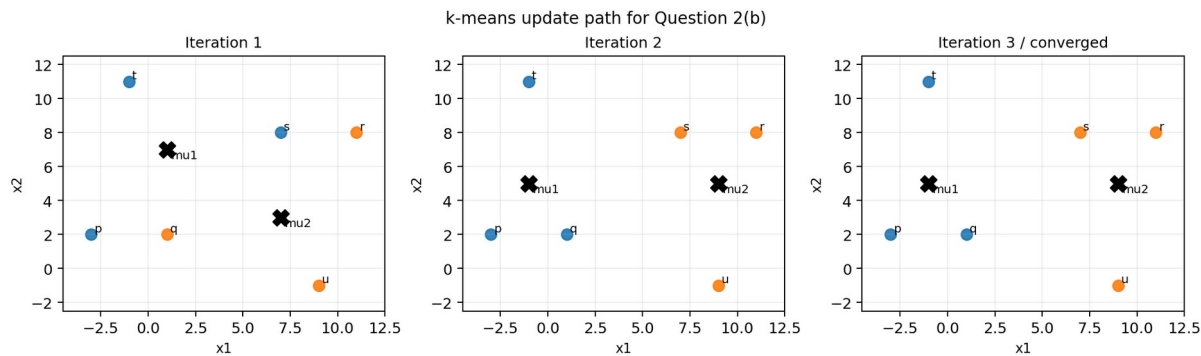


Illustration: cluster memberships settle after the second update, so the third pass confirms convergence.

2(c) Two limitations of k-means

- The number of clusters k must be chosen in advance. If k is poorly chosen, the result can be misleading even when the optimization is done correctly.
- The algorithm assumes the mean is a sensible representative and works best for roughly spherical clusters under Euclidean distance. It struggles with elongated, non-convex or very unequal-density clusters, and it is also sensitive to initialization.

2(d) A more outlier-robust modification of k-means

A simple and effective modification is trimmed k-means. The algorithm still alternates between assignment and update, but after assigning every point to its nearest centroid, we compute the distance of each point to its assigned centroid and temporarily ignore the few most extreme points when recomputing the means.

- Step 1: assign each point to the nearest centroid as usual.
- Step 2: within each cluster, rank the points by distance from the centroid; either discard the top τ percent or give them a very small weight.
- Step 3: recompute each centroid using only the retained / down-weighted points.

Why it helps: in ordinary k-means, a single far-away point can pull the mean strongly. Trimming or down-weighting caps that influence, so the centroid is determined mainly by the dense core of the cluster. The final omitted points can then be flagged as outliers, or assigned only after the robust centroids have stabilized.

Question 3 - Evaluation and Combining Learners

3(a) Why 97% accuracy on a binary task is not enough by itself

Accuracy must be interpreted in context. At least the following extra information is needed before judging whether 97% is good.

- Class balance / baseline rate. If 97% of the examples belong to one class, a trivial majority-class classifier already achieves 97% accuracy, so the result would not be impressive. If the classes are balanced, 97% would be much stronger.
- Error costs and confusion-matrix structure. In many real problems, false negatives and false positives have different consequences. Two systems can have the same 97% accuracy but very different recall or precision on the important class.
- How accuracy was estimated. One needs to know whether it came from a proper test set or cross-validation, and on how many examples. An accuracy of 97% on 100 examples is less convincing than 97% on 100000 examples because the uncertainty is much larger.

3(b) Confusion matrix calculations

From the table, the entries are read as follows:

TP	FN	FP	TN	Total
72	18	8	32	130

The estimated prior probability of class Yes is the proportion of actual Yes instances:

$$P(\text{Yes}) = (TP + FN) / N = (72 + 18) / 130 = 90 / 130 = 0.6923.$$

Recall (true positive rate) measures how many actual Yes cases are recovered:

$$\text{Recall} = TP / (TP + FN) = 72 / (72 + 18) = 72 / 90 = 0.8000.$$

Precision measures how many predicted Yes cases are correct:

$$\text{Precision} = TP / (TP + FP) = 72 / (72 + 8) = 72 / 80 = 0.9000.$$

Classification accuracy is the proportion of correct predictions overall:

$$\text{Accuracy} = (TP + TN) / N = (72 + 32) / 130 = 104 / 130 = 0.8000.$$

3(c) A co-operative method for combining several partially successful learning procedures

A strong way to make several different learners co-operate is stacked generalization (stacking). The idea is not just to average them blindly, but to train a second-level model that learns when to trust each base learner.

- Step 1: split the training data into K folds.
- Step 2: for each base learner and for each fold, train on K-1 folds and predict the held-out fold. This produces out-of-fold predictions for every training example.
- Step 3: use those out-of-fold predictions (for example class probabilities from all learners) as the input features for a meta-learner such as logistic regression.
- Step 4: retrain each base learner on the full training set. At test time, obtain their predictions on the new case and feed those predictions to the meta-learner for the final decision.

Why this can improve accuracy: different learning procedures often make different errors. The meta-learner can detect patterns such as “trust the tree on region A of the feature space, but trust the SVM on region B” or “when models 1 and 3 agree strongly, their consensus is reliable”. Because the second stage is trained on out-of-fold predictions rather than in-sample predictions, it reduces information leakage and uses the complementary strengths of the component models.

Final numerical results used in the worked solutions: Question 2(b): final centroids (-1,5) and (9,5). Question 3(b): prior $P(\text{Yes})=0.6923$, recall=0.8000, precision=0.9000, accuracy=0.8000.